AOSP 进阶丛书 · 第十一册 · ANDROID 13 (API 33)

存储与加密密钥全链

FBE 双域与 FsCrypt 逐段、密钥链五层（Gatekeeper→Synthetic Password→文件密钥）、keystore2 与 KeyMint 硬件仓库、direct boot 时间线、vold 卷管理与信任链四道墙收束。

源码基线：AOSP Android 13 (TP1A · SDK 33) · 行号全部实测核验

目 录

前言：数据的一生：加密、密钥与信任 2
第一章 FBE 双域：/data 的两把钥匙 2
第二章 密钥链：从手指到文件密钥 3
第三章 keystore2：Rust 重写的密钥仓库 4
第四章 direct boot：开机到解锁的时间线 5
第五章 vold：卷与存储的全职管家 6
第六章 信任链收束：四道墙拼图 6
第七章 设计决策九问 7
附录 A 观测实操 8
附录 B 快问快答（16 条） 8
附录 C 自检题 9

本册速览

FBE 双域：DE(开机即解)与 CE(解锁后解)——FsCrypt.cpp 逐段
密钥链五层：PIN→Gatekeeper→Synthetic Password→DE/CE Key→文件密钥
keystore2(Rust)与 KeyMint(TEE/StrongBox)·attestation 证书链
direct boot 时间线与 vold 卷管理(FUSE/Checkpoint)
信任链四道墙收束：BootROM/AVB/dm-verity/fscrypt+KeyMint

前言：数据的一生：加密、密钥与信任

前十册讲"代码怎么跑"，本册讲"**数据怎么被保护**"：/data 上每个文件属于哪个加密域（FBE）、保护它的密钥从哪来到哪去（密钥链/KeyMint）、App 的私钥住在哪（keystore2）、重启后到解锁前的世界什么样（direct boot）、这一切与 AVB/dm-verity 如何拼成完整信任链。

源码基线不变（本树 Android 13）。主角目录：`system/vold`（FsCrypt/KeyStorage/Keymaster/VolumeManager）、`system/security/keystore2`（**Rust** 新世界）、`hardware/interfaces/keymint`（AIDL HAL）、`frameworks/base/services/core/.../locksettings`（LockSettingsService:202）。

第一章 FBE 双域：/data 的两把钥匙



图 1 FBE 双域：DE（开机即解）与 CE（解锁后解）的目录/住客/密钥来源，FsCrypt.cpp:842-873 逐段

1.1 逐段注释（真实源码）

cpp system/vold/FsCrypt.cpp:842-873（节选+讲解注释）

```
// 讲解注释①：BuildDataUserDePath(volume_uuid, user_id)（:842）
// ——DE 域目录寻址：/data/user_de/<id>。开机即挂（无需口令）。
auto user_de_path = android::vold::BuildDataUserDePath(volume_uuid, user_id);
// 讲解注释②：prepare_dir(:858)：建目录并落权限（0771 system:system）
// ——目录本身的可见性由 Unix 权限管，内容保密由 fscrypt 管。
```

```
if (!prepare_dir(user_de_path, 0771, AID_SYSTEM, AID_SYSTEM)) return false;
// 讲解注释③：EnsurePolicy(de_policy, user_de_path)（:873）
// ——给目录绑 fscrypt 加密策略：此后该目录下每个新文件
// 由内核按策略密钥独立加密（每文件 HKDF 派生子密钥）。
if (!EnsurePolicy(de_policy, user_de_path)) return false;
// CE 侧同构（user/0 目录），但装钥要等解锁（第六章 direct boot）。
```

设计洞察 · 逐段精读：fscrypt 与 FDE 的世代差

FDE（全盘加密，EncryptInplace.cpp 的活化石）整盘一把钥匙、解锁前全系统不可用；FBE（文件级）按目录一域一钥——**重启后闹钟照响（DE）而微信看不到（CE）**成为可能。这就是 Android 7.1 以来 direct boot 的物理基础。

第二章 密钥链：从手指到文件密钥



图 2 密钥链五层：屏幕凭据→Gatekeeper→Synthetic Password→DE/CE Key→文件密钥；改 PIN 不重加密的秘密

2.1 链条逐层（含 LockSettingsService）

text 密钥链五层（路径+锚点）

```
① 屏幕凭据：PIN/图案/密码（+生物特征=另一把钥匙）
② Gatekeeper HAL（hardware/interfaces/gatekeeper/1.0，冻结维护）
——TEE 内校验+防暴力；框架只见"对/错/超时"签名
③ Synthetic Password（LockSettingsService.java:202 域）
——框架级"万能中间凭据"：CE key 由 SP 派生；
PIN/指纹都只是"解锁 SP 的钥匙"→ 改 PIN 不重加密全盘
```

④ DE/CE Key（vold KeyStorage.cpp；FsCrypt.cpp:195 KeyAuthentication）
——DE key 由硬件绑定 KMK 包装（无需口令）；CE key 由 SP 派生
⑤ 文件密钥：内核 fscrypt 按目录密钥 HKDF 派生（每文件独立）

> **设计洞察 · 面试核心：改密码为什么不用重加密**
> CE key 不随 PIN 变——变的是"SP 的包装"（KEK 双份：Gatekeeper 签名密钥+Weaver）。同理注册新指纹=给 SP 加一把钥匙，/data 毫无察觉。**间接层（SP）是密码学工程里的'稳定器'**。

2.3 密钥的落点与备份（数据侧最后一问）

"换机/恢复出厂密钥怎么办"三答案：**CE/DE 密钥永不迁移**（换机搬家的是密文+云端重下，不是密钥）；**keystore2 密钥不可导出**（KeyMint 硬件绑定——所以银行要在新机重新绑定）；**synthetic password 不参与云备份**（PIN 只在本机）。云备份（书6 § 三）备份的密文用云端密钥重加密——**两条密钥链（本机链/云链）互不相交**是"云备份可用性与本机安全"并存的设计底座。

2.2 防暴力与 StrongAuth

Gatekeeper 错误 N 次→StrongAuth 锁定（LockSettingsStrongAuth）：**重启后必须输 PIN（生物禁用）**的规则本源；Weaver（部分设备）是防离线爆破的硬件计数器。整个设计 TEE 边界清晰：**PIN 本体永不离开框架的加密包装**。

第三章 keystore2：Rust 重写的密钥仓库



图 3 keystore2 三层：App API→keystore2（service.rs:54 按安全级路由）→KeyMint HAL（TEE/StrongBox）

```
text system/security/keystore2（Rust）三层结构
App: android.security.keystore2.KeyGenParameterSpec
→ generateKey（App 只有句柄，无私钥本体）
keystore2: service.rs:54 i_sec_level_by_uuid（按 KeyMint 实例路由）
database.rs（条目+授权+审计） · async_task（队列） · attestation（证书链）
KeyMint HAL（hardware/interfaces/keymint/aidl）：
TEE 或 StrongBox（独立安全芯片）实现；
密钥永不出硬件；操作（签名/解密）进硬件执行
```

三条设计点：**句柄而非密钥**（App 侧导不出私钥——银行 App 的安全底气）；**授权三层**（条目 grant/用法 KeyGenParameterSpec/用户认证 setUserAuthenticationRequired——与书6 权限三层同构的思想）；**attestation 证书链**（密钥生成时硬件签名证明"这密钥在这设备的安全硬件里"——Play Integrity 的信任根）。观测 `dumpsys keystore2`。

第四章 direct boot：开机到解锁的时间线



图 4 Direct Boot 时间线：t0 开机→t1 DE 解（vold 无口令装钥）→t2 锁屏→t3 输对凭据→t4 CE 解（USER_UNLOCKED 广播）

App 适配两件事：`directBootAware=true` 的组件在 DE 期即跑（只给闹钟/电话这类最小集）；常规 App 监听 ACTION_BOOT_COMPLETED 与 ACTION_USER_UNLOCKED **两个**广播分别处理。CE 未解时访问其数据是"挂起"不是崩溃。经典 bug：开机后通知点击打开 App 却是空数据=没处理解锁前空态。

第五章 vold：卷与存储的全职管家



图 5 vold 四类卷生命周期（VolumeManager:173/:442）、FUSE 与 scoped storage、OTA 数据面 Checkpoint

vold（main.cpp:65；VolumeManager::start:173）是 /data 相关一切的唯一写者：FBE 装钥（前两章）、四类卷（private/public/adoptable/stub）、**FUSE 代理**——11 起 /storage 经 MediaProvider 进程审计改写（书6 scoped storage 的物理执行；13 的 passthrough 直通是性能回归）。**Checkpoint.cpp** 是 OTA 的数据面保险：与 A/B 的系统面（书2 § 十一）成对。客户端 `vdc/sm` 命令族。

第六章 信任链收束：四道墙拼图



图 6 信任链全景：BootROM→AVB（书2）→dm-verity（书6）→fscrypt/KeyMint（本册），各防什么与解锁后剩什么

四层职责不重叠：**防刷入（BootROM 熔丝）、防启动链篡改（AVB）、防运行时改系统（dm-verity）、防数据提取与密钥泄露（fscrypt+KeyMint）**。解锁 bootloader 后 AVB 橙警、verity 可禁，但 fscrypt 照常、KeyMint 照常（TEE 不看锁）——attestation 会标注解锁态，Play Integrity 据此识别。

第七章 设计决策九问

- **为什么 FBE 取代 FDE？** 粒度：按域解密让"重启后部分可用"成立；FDE 全有或全无，体验死锁。
- **为什么引入 Synthetic Password？** 换凭据不重加密（性能不可能每次全盘）；且支持"多把钥匙开同一扇门"（PIN+指纹）。
- **为什么 DE key 不需要口令？** 开机即用的服务（闹钟）无凭据可验——DE 的威胁模型本来就是"物理窃取后系统级保护"，KMK 硬件绑定足够。
- **为什么 keystore2 用 Rust 重写？** 并发安全（密钥操作的竞态代价=安全事故）；旧 Java keystore1 的授权模型僵硬（不能按时段/按用途细粒度）。
- **为什么 KeyMint 分 TEE/StrongBox 两级？** 成本与安全分级：StrongBox 独立芯片抗物理攻击更贵，高价值密钥（支付）才配。
- **为什么 attestation 是证书链不是单证书？** 设备根→中间→密钥三级可吊销可轮换（与 Web PKI 同构）。
- **为什么 FUSE 而不是内核原生 scoped storage？** 审计逻辑高频迭代（Mainline 可升级）；内核逻辑只能跟整机 OTA。

- **为什么 OTA 数据面要 Checkpoint?** A/B 只保系统面；应用数据被半 OTA 弄坏同样灾难——数据面快照补齐"全有或全无"。
- **为什么 Gatekeeper 冻结在 HIDL 1.0?** 接口极简（verify/enroll）无演进需求；迁移成本>收益（冻结也是架构决策）。

附录 A 观测实操

目标	命令/工具	看点
加密状态	dumpsys storageencryption	FBE 模式/密钥状态
vold 客户端	vdc cryptfs / vdc volume	shell 侧主力
keystore	dumpsys keystore2	条目/授权/审计
解锁状态	getprop sys.user.0.ce_available	CE 解锁事实
direct boot	dumpsys user	用户解锁状态
磁盘	sm list-disks / sm list-volumes	adoptable/storage
锁屏策略	cmd lock_settings get-disabled	StrongAuth 状态

附录 B 快问快答（16 条）

- FBE 与 FDE 一句话区别？——按目录分钥（域级）vs 全盘一钥。
- DE/CE 目录分别是什么？——/data/user_de/N 与 /data/user/N。
- 改 PIN 后 CE key 变吗？——不变（SP 间接层）。
- 指纹解锁的密码学含义？——SP 的另一把钥匙。
- Gatekeeper 在哪跑？——TEE/独立核（HAL 1.0 冻结）。
- keystore2 什么语言？——Rust（service.rs:54 路由 KeyMint）。
- StrongBox 与 TEE 区别？——独立安全芯片（更高抗物理，更贵）。
- attestation 给谁用？——远程证明（Play Integrity/银行）。
- 解锁前微信数据可见吗？——不可见（CE 未解，访问挂起）。
- USER_UNLOCKED 与 BOOT_COMPLETED 区别？——CE 解锁 vs 系统就绪（前者更晚）。
- FUSE 守门人是谁？——MediaProvider 进程（13 passthrough 优化）。
- adoptable 存储是什么？——SD 卡并入加密池变内部存储。
- OTA 的数据面保险？——vold Checkpoint（与 A/B 成对）。
- 解锁 bootloader 后 fscrypt？——仍生效（密钥随用户数据）。
- 四道信任墙？——BootROM/AVB/dm-verity/fscrypt+KeyMint。

- 本册之后？——书12 媒体（MediaDrm 用 KeyMint 分支）。

附录 C 自检题

- 默画密钥链五层与改 PIN 时的哪层变化。
- FsCrypt 三个锚点（:842/:858/:873）各做什么？
- direct boot 时间线五点；App 双广播适配。
- keystore2 三层与"导不出私钥"的机制保证。
- 解锁 bootloader 后四道墙各剩几道？